# LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Daycare Fees Application | Hypothesis Property-Based Testing

| | |
|---|---|
| **Student** | Arnav Choudhary |
| **Repository** | SETAPESU26/L6_L7_48_daycare_fees |
| **Lab folder** | Lab 7 |
| **Environment** | Python 3.12.4 |
| **Testing tools** | Hypothesis 6.170.15 and pytest 9.2.0 |

## Objective

To explore fuzz testing as a security and quality assurance technique by using the Hypothesis library to generate unexpected inputs, identify input-validation bugs in a Python application, and improve robustness through defensive programming.

### Application functions under test

**parse_enrolment()** parses an age group and hours; **hours_average()** calculates the average of comma-separated weekly hours; and **payable_fee()** calculates the payable fee after tax and discount.

# 1. Environment Setup

The repository was cloned locally. A virtual environment was created and activated, and the required testing dependencies were installed.

```
python3 -m venv .venv
source .venv/bin/activate
pip install hypothesis pytest
```

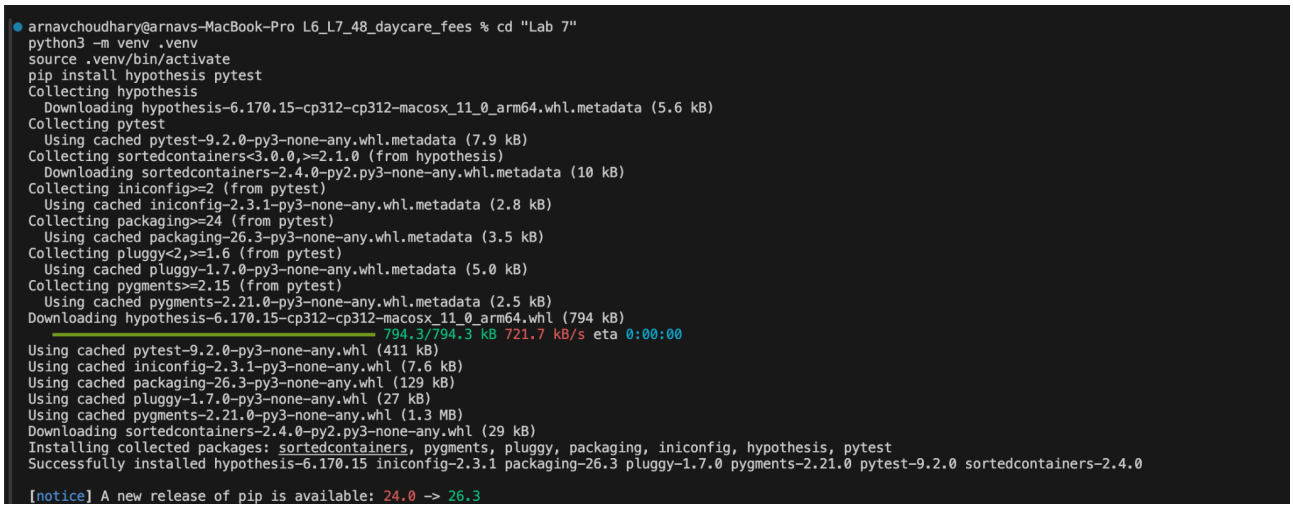



Figure 1. Virtual environment setup and installation of Hypothesis and pytest.

# 2. Original Application Execution

The original program was first tested with an invalid enrolment value (25). It crashed because the function expected two fields separated by a colon. The program was then run with valid sample inputs and produced the expected outputs.

```
Valid inputs and outputs:
toddler:25  -> ('Toddler', 25.0)
20,25,30    -> 25
3000|250    -> 3290.0

Invalid input 25 caused:
ValueError: not enough values to unpack (expected 2, got 1)
```

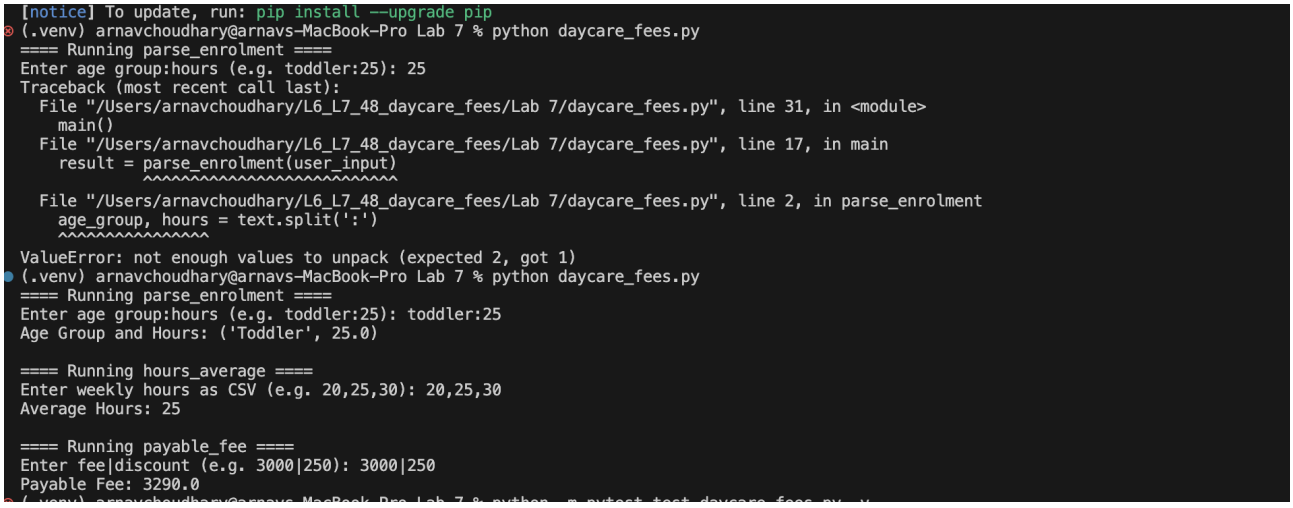



Figure 2. Original application: invalid enrolment input causes a ValueError, followed by successful execution with valid inputs.

# 3. Hypothesis Fuzz Testing – Initial Failures

Three tests were written using the @given decorator and strategies that generate arbitrary text or None values. Hypothesis discovered failing inputs for all three functions.

| Function | Examples of failing inputs | Problem found |
|---|---|---|
| parse_enrolment() | ':' , '' , None | Missing/empty fields and invalid input type |
| hours_average() | '' , None | Empty numeric field and invalid input type |
| payable_fee() | '\|' , '' , None | Missing/empty fields and invalid input type |

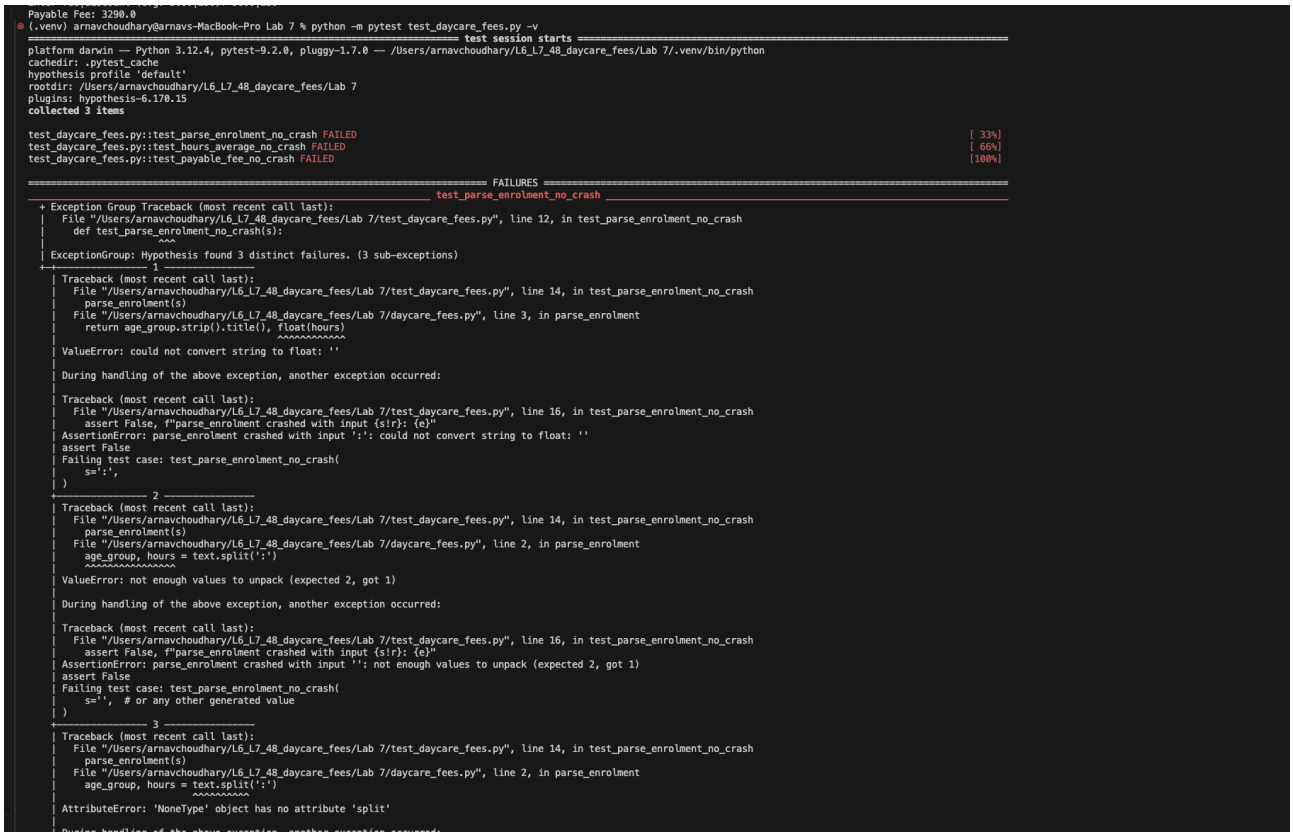



Figure 3. Initial Hypothesis fuzz tests: all three tests fail and expose input-validation defects.

# 4. Fixes Applied

The three functions were hardened with defensive input validation. The corrected implementation checks input types, validates separators and field counts, catches numeric conversion errors, rejects negative and non-finite numeric values, and returns safe fallback values for invalid inputs.

### Fallback behaviour for invalid input

```
parse_enrolment(invalid) -> ('Unknown', 0.0)
hours_average(invalid) -> 0
payable_fee(invalid) -> 0.0
```

# 5. Verification After Fixing

The corrected application was tested with both invalid and valid input. Invalid values did not crash the program, while valid sample values retained their expected results.

```
Valid inputs:
toddler:25  -> ('Toddler', 25.0)
20,25,30    -> 25
3000|250    -> 3290.0

Invalid inputs used in the run:
25, . , / -> safe fallback outputs
```

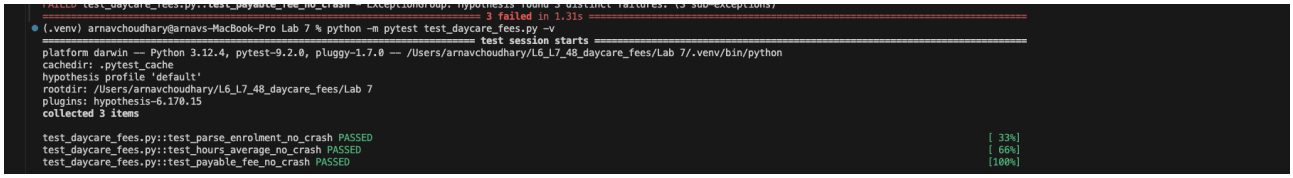



Figure 4. Rerun of Hypothesis tests after fixing the application: all three tests pass.

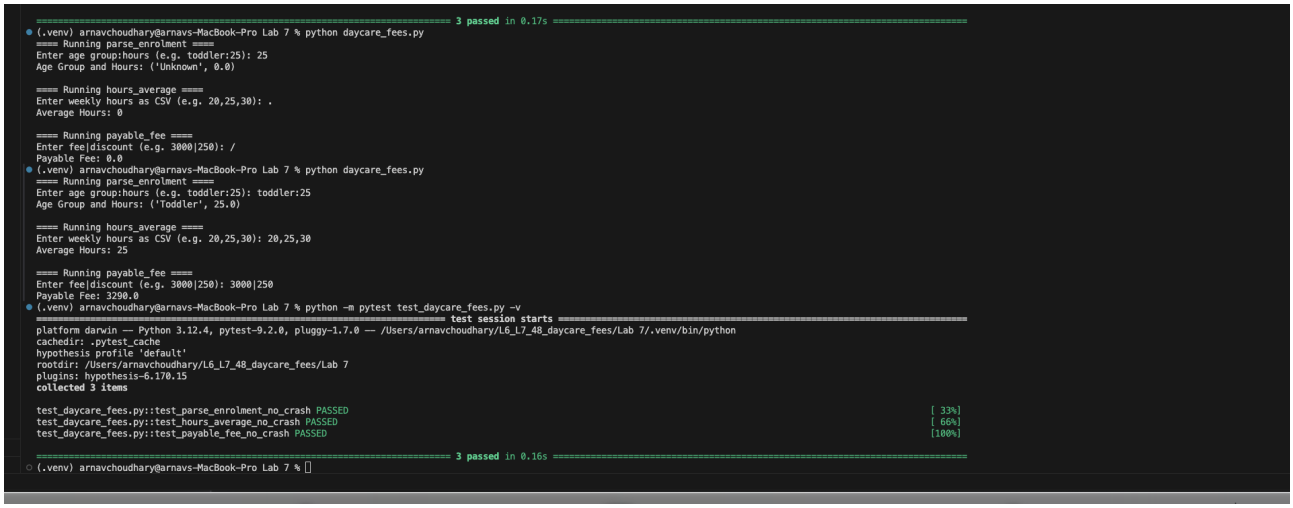



Figure 5. Corrected application handles invalid and valid inputs, followed by the final passing test run.

# 6. Final Test Result

```
test_parse_enrolment_no_crash  PASSED
test_hours_average_no_crash    PASSED
test_payable_fee_no_crash      PASSED

3 passed
```

# 7. Conclusion

Hypothesis fuzz testing exposed input-validation defects in all three functions. After defensive validation was added, the application handled the demonstrated invalid inputs without crashing and continued to return the expected results for valid sample inputs. All three no-crash fuzz tests passed after the fixes.

Submission note: The fixed daycare_fees.py and updated test_daycare_fees.py should be submitted separately alongside this PDF report.